

Feature Suggestions for Manthan Agent

Eight capability areas identified by comparing recurring patterns across a survey of AI-assistant skill catalogs, matched against Manthan Agent's current feature set (multi-provider chat, MCP tools, file export, Python execution, and live app previews).

Each entry includes: why it matters, what "done" looks like, an implementation sketch for Manthan Agent's own architecture, and an original skill-definition body you can drop into your own skills/instructions system.

Compiled 25 September 2026

Note: this document is written fresh for Manthan Agent. It does not reproduce internal skill text from any other AI product — only original explanations and implementation guidance based on general, publicly-known patterns.

Contents

1. Persistent Memory & Context
2. Structured File QA Loop (DOCX / PPTX / XLSX)
3. PDF Form Filling with AcroForm-First Strategy
4. Connected Apps & Plugin Integrations
5. Automations & Scheduled Tasks
6. Explicit Safety & Parental Controls Surface
7. Entity & Citation-Rich Responses
8. Workflow Router for Complex Requests

Persistent Memory & Context

Why it matters

Right now Manthan Agent (per its incognito toggle and per-session chat state) treats each conversation independently. Users repeat context — their name, their stack, their preferences — every session. A memory layer removes that friction and makes the assistant feel like it "knows" the user over time.

What “done” looks like

- A per-user memory store persists across sessions, separate from chat history.
- The model reads relevant memory at the start of a session and writes new durable facts as they emerge.
- Users can view, edit, and delete what's stored, from Settings.
- Memory is scoped per-account, never leaks between users, and is skipped entirely in Incognito Mode.

How it fits Manthan Agent's architecture

Manthan Agent already has a client-side `state` object and IndexedDB/localStorage-style persistence patterns (seen in the app's storage helpers). Memory should NOT live only in the browser — it needs a small backend table (or a Postgres/SQLite table if you already run a backend for auth) keyed by `user_id`, holding short markdown-style fact fragments. On each new session, fetch the user's memory blob, inject it into the system prompt as a bounded-size context block, and expose an internal tool the model can call to append/update/delete a fact. Cap total memory size and periodically ask a cheap model call to compress/deduplicate it.

Skill definition (drop-in)

```
---
name: user-memory
description: Maintain a durable, per-user memory profile across sessions. Use this whenever the user states a stable fact about themselves (name, role, preferences, recurring projects) or when a response would benefit from recalling previously stated facts. Never used in Incognito Mode.
---

## When to write
- The user states a fact that would plausibly still be true in a few months (their job, tech stack, recurring project, stated preference for tone/format/language).
- Do NOT write transient details (today's task, current mood, a one-off question).

## When to read
- At the start of every non-incognito session, load the user's memory block and use it only where it changes the substance of a reply — never as decoration or forced personalization.

## Storage shape
One short markdown file per user, organized under light headings (Profile, Preferences, Projects). Each fact is a single bullet line. Keep the whole file under ~2KB; when it grows past that, summarize and merge overlapping lines rather than appending indefinitely.
```

Privacy rules

- Never store payment details, government ID numbers, passwords, or health/legal specifics unless the user explicitly asks and confirms they understand it will persist.
- Any deletion request must be honored immediately and completely – no soft-delete that the model can still see.
- Memory is never shared across user accounts, and is fully disabled when Incognito is on.

Structured File QA Loop (DOCX / PPTX / XLSX)

Why it matters

Manthan Agent already generates DOCX, PPTX, and XLSX files client-side via docx.js, pptxgenjs, and SheetJS. What it doesn't do yet is verify the result before handing it to the user — text can overflow a slide, a table can break across an unexpected page, fonts can silently fall back. A QA pass catches this before the user downloads a broken file.

What “done” looks like

- After generating a document/deck/sheet, the file is checked programmatically for common defects.
- Overflowing text boxes, truncated table cells, and missing fonts are detected automatically.
- Where possible, the layout is auto-corrected (shrink font, resize box) and re-checked.
- If an issue can't be auto-fixed, the user is told plainly what to expect before download.

How it fits Manthan Agent's architecture

Since generation happens client-side in the browser (no Python/LibreOffice available at runtime), full pixel-level QA isn't feasible without a server step. The practical approach: (1) do structural validation in JS right after generation — for pptxgenjs, check each text box's estimated content height against its box height using a canvas-based text-measurement helper before committing font size; for docx.js, validate that table cell character counts stay under a safe threshold for the column width; (2) optionally add a lightweight server endpoint (or a headless LibreOffice worker) that converts the generated file to page images for a final visual check on higher-stakes exports; (3) surface a short 'QA passed / N issues auto-fixed' note in the export toast/notification.

Skill definition (drop-in)

```
---
name: document-qa-loop
description: Run automated quality checks after generating a DOCX, PPTX, or XLSX file, before presenting it to the user. Use immediately after any client-side document/deck/sheet generation.
---

## Checks to run
1. Text overflow – estimate rendered text height/width against the container; if it would overflow, first try reducing font size within an allowed range, then try shortening spacing, and only then flag it as unresolved.
2. Table integrity – verify no cell's content silently truncates; widen the column or wrap text before shrinking font below a readability floor.
3. Font fallback – confirm every font family used is either embedded or in the standard supported set; substitute a close, embedded alternative rather than leaving a fallback that differs across viewers.
4. Structural consistency – for decks, confirm slide count and titles match what was requested; for sheets, confirm formulas reference existing ranges, not stale ones.
```

Auto-fix vs. flag

Attempt an automatic fix first for any check above. If a fix cannot be applied confidently (e.g., content genuinely too long for any reasonable font size), leave the content as-is and clearly tell the user which slide/page/cell may need manual trimming – never silently ship a broken layout without disclosure.

Output

Report a one-line summary alongside the file: either "Checked – no issues found" or "Checked – auto-fixed 2 issues, 1 needs your review (slide 4 text runs long)."

PDF Form Filling with AcroForm-First Strategy

Why it matters

Users frequently upload a PDF and ask to "fill this out." Many such PDFs are proper fillable forms (AcroForm fields); others are flat scans or exports with no fields at all. Handling both cases — and choosing correctly between them — is a distinct, high-value capability separate from general PDF export.

What “done” looks like

- Uploading a fillable PDF lets the user (or the assistant) populate its actual form fields.
- If the PDF has no form fields, the assistant falls back to placing text at the correct visual position.
- The filled result can be flattened into a final, non-editable PDF on request.
- Field detection and filling happens without corrupting the PDF's existing structure or other content.

How it fits Manthan Agent's architecture

Manthan Agent already loads jsPDF and jszip client-side, but neither reads existing AcroForm structures. This needs a PDF-parsing library capable of reading and writing form fields client-side (pdf-lib is the standard choice and is MIT-licensed, loadable from a CDN like your other libraries). Flow: (1) on PDF upload, use pdf-lib to call ``getForm()`` and enumerate fields; (2) if fields exist, map the user's answers (via chat) to field names and set values directly — this preserves the original PDF's exact appearance; (3) if no fields exist, fall back to ``page.drawText()`` at coordinates either supplied by the user or inferred by a rough text-layout heuristic; (4) offer a 'flatten' step using ``form.flatten()`` before final export.

Skill definition (drop-in)

```
---
name: pdf-form-fill
description: Fill out PDF forms uploaded by the user. Detects real AcroForm fields first and fills them directly; only falls back to positioned text overlay when the PDF has no fillable fields. Use whenever a user uploads a PDF and asks to fill it, complete it, or put information into it.
---

## Step 1 – detect
Open the uploaded PDF and check for an AcroForm field dictionary. If present, list every field name, type (text, checkbox, radio, dropdown), and current value back to the user (or infer values from what they've already told you in chat) before filling.

## Step 2a – native fill (preferred path)
Set each field's value directly through the form API. This preserves the original PDF's fonts, layout, and appearance exactly, and keeps the file editable until flattened. Never guess at field names – use the ones actually present in the document.

## Step 2b – overlay fallback (no fields present)
Only used when Step 1 finds zero fields. Determine likely label positions from the extracted text layout, then draw the requested values at those coordinates in a plain, legible font. Tell the use
```

r this is an approximation and ask them to verify alignment before relying on it for anything official.

Step 3 – flatten (optional)

On request, merge field values permanently into the page content so the form can no longer be edited – appropriate once the user confirms the filled form is final.

Safety notes

- Never fabricate values for fields the user hasn't provided; leave them blank and ask.
- Flag if a form appears to require a wet signature, notarization, or legal witness – filling text fields doesn't substitute for those requirements.

Connected Apps & Plugin Integrations

Why it matters

Manthan Agent supports MCP tools, which is the right foundation, but MCP configuration is still a technical, JSON-based setup most end users won't do themselves. A first-class integrations panel — "Connect your Calendar", "Connect Google Drive" — turns the same underlying MCP capability into something a non-technical user can enable in two clicks.

What “done” looks like

- A visible Integrations/Connections panel lists common services with a one-click Connect button.
- OAuth handles the actual authentication; Manthan Agent never sees or stores raw passwords.
- Connected/disconnected state is shown inline in chat when a relevant request comes up.
- A single user message can trigger a chain of calls across more than one connected app.

How it fits Manthan Agent's architecture

This sits on top of your existing MCP tool-calling support rather than replacing it: each "integration" is really a pre-configured MCP server (or a thin proxy you host) with a friendly name, icon, and OAuth flow baked in, instead of requiring the user to paste a server URL and token manually. Build: (1) a small backend OAuth broker per supported service that stores refresh tokens server-side, never in the browser; (2) a settings panel that lists available integrations and reflects live connect/disconnect state; (3) surface a lightweight suggestion UI in chat (similar in spirit to your existing tools menu) offering the relevant connected app when a user's request matches its category, rather than silently guessing which service to use.

Skill definition (drop-in)

```
---
name: connected-app-suggestion
description: Offer a relevant connected app (calendar, drive, email, etc.) when the user's request
  implies reading or acting on their external data and no tool is already available for it. Use before
  falling back to asking the user to paste information manually.
---

## When to suggest
The user's request implies personal/external data – "what's on my calendar", "find that file I shared
last week", "did I get a reply from them" – and no connected tool currently covers it.

## How to suggest
Show the matching integration(s) with a short, specific line about what calling it would do ("I can
check your Calendar for tomorrow's meetings") rather than a generic "connect an app" prompt. Never
guess which of several similar services (e.g., which calendar provider) the user means if more
than one is plausible – ask, or offer both.

## After connecting
Once authorized, call the tool directly for this and future matching requests in the session witho
```

ut re-prompting, unless the user disconnects it.

Chaining

A single user request may need more than one connected app (e.g., "find the invoice email and add the due date to my calendar") – call each tool in sequence within one reply rather than requiring separate turns.

Privacy

Never display or log raw OAuth tokens or credentials in chat. Only reflect connect/disconnect status and the data the user's own request requires.

Automations & Scheduled Tasks

Why it matters

Every interaction with Manthan Agent today is user-initiated. A scheduling layer — "send me a morning briefing every weekday at 8am" — turns the assistant from a reactive tool into something that proactively delivers value, which is a strong retention feature.

What “done” looks like

- A user can turn any prompt into a recurring job with a plain-language or cron-style schedule.
- Scheduled runs execute server-side (not dependent on the browser tab being open).
- Results land as a new chat message or a push/in-app notification, not lost if unread.
- Users can view, pause, edit, and delete their automations from a dedicated panel.

How it fits Manthan Agent's architecture

This is the one feature here that genuinely requires a backend job runner, since a browser tab can't reliably fire a request at 8am if closed. Needed pieces: (1) a `scheduled_tasks` table (`user_id`, `prompt`, `schedule`, `model/tools config`, `next_run_at`, `active`); (2) a lightweight cron worker (or a managed scheduler like a cloud provider's cron jobs) that polls due tasks and replays them through the same completion pipeline the chat UI uses; (3) a delivery path — write the result into a 'automation run' conversation thread and optionally push a web/mobile notification; (4) a management UI reusing your existing sidebar/panel patterns.

Skill definition (drop-in)

```
---
name: scheduled-automation
description: Convert a user's request into a recurring scheduled task, and manage existing automations (view, pause, edit, delete). Use when the user asks for something to happen repeatedly, on a schedule, or "every day/week" without them re-prompting.
---

## Creating an automation
Confirm three things before saving: (1) the exact prompt/task to run each time, (2) the schedule in plain terms ("weekdays at 8am", "every Monday"), and (3) where the result should go (a chat thread, a notification, or both). Restate the confirmed schedule back to the user before saving.

## Running an automation
Each run executes the saved prompt fresh, with current data and tools – never a cached result. If a run depends on a connected app that's since been disconnected, skip the run and notify the user rather than failing silently.

## Managing automations
Support pausing (keeps the schedule but skips runs), editing (schedule or prompt), and deleting outright. Always show the user their current list of active automations on request, not just when creating a new one.
```

Guardrails

- Cap the number of active automations per user to prevent runaway resource use.
- Never schedule a task that would repeatedly perform an irreversible action (sending money, deleting data) without an explicit per-run confirmation step.

Explicit Safety & Parental Controls Surface

Why it matters

If Manthan Agent is used by families or in managed accounts, safety settings that only exist as hidden backend logic are hard to trust and hard to audit. Making the current restriction state visible — and, where appropriate, adjustable by an account owner — builds trust and meets a bar many users now expect from AI products.

What “done” looks like

- Account owners can see what content restrictions currently apply to a managed/child account.
- Adjusting a restriction takes effect immediately and is confirmed in the UI.
- Blocked actions are logged (what was blocked, when) for the account owner to review.
- The restriction state is readable by the assistant itself, not just enforced silently.

How it fits Manthan Agent's architecture

This is primarily a UI + audit-log feature layered on whatever content-safety logic already runs (client-side or via your model provider's own safety filtering). Add: (1) a family/account-linking model if one doesn't already exist (owner account ↔ managed account); (2) a settings surface showing current restriction level and any per-category toggles; (3) an append-only log table for blocked-request events, written whenever a request trips a restriction, shown to the owner with a timestamp and category (not full message content, to preserve the managed user's privacy — general category is usually enough).

Skill definition (drop-in)

```
---
name: safety-controls-transparency
description: Read and respect the current account's safety/content-restriction settings, and log when a request is blocked because of them. Use on every request for a managed account, and whenever a user asks what restrictions are currently active.
---

## Reading restriction state
Before responding, check whether the current account has active content restrictions. If so, apply them in addition to (never instead of) baseline safety behavior — restrictions can only narrow what's allowed, never widen it.

## When a request is blocked by a restriction
Tell the managed user plainly that this type of request isn't available on their account, without being preachy or over-explaining. Log the event (category only, not full message content) so the account owner can review it later.

## Owner-facing queries
When an account owner asks what's currently restricted, state the settings plainly and accurately — don't guess or approximate. Never reveal a managed user's actual conversation content when discussing settings with the owner; only the restriction categories and blocked-event log apply.
```

What this skill never does

Never silently apply restrictions that aren't actually configured, and never remove or loosen a restriction without an explicit, authenticated action from the account owner.

Entity & Citation-Rich Responses

Why it matters

Manthan Agent already does OpenRouter-based web search. Raw links in plain text are easy to skim past. Turning named things (people, companies, places, products) into recognizable, previewable references — and visually separating true citations from ordinary hyperlinks — makes search-backed answers noticeably easier to scan and trust.

What “done” looks like

- Named entities mentioned in a response are visually distinct and show a brief preview on hover/tap.
- Citations backing a specific claim look different from a plain inline link.
- Search results that are images or videos render as a small carousel instead of a bare list of links.
- None of this slows down normal text-only replies where it isn't relevant.

How it fits Manthan Agent's architecture

This is a rendering-layer feature on top of your existing markdown/message-rendering pipeline (DOMPurify is already in your stack for sanitizing model output, which is the right place to also do this). Approach: (1) have the model emit a lightweight structured marker for entities and citations in its raw output (e.g., a custom markdown extension or a JSON sidecar block), (2) a post-processing pass in the frontend converts these markers into styled spans/chips with a small popover component, (3) reuse image/video search results you already fetch to populate a carousel component instead of a plain list. Keep this strictly additive — fall back to plain links whenever a marker is missing or malformed, so rendering never breaks.

Skill definition (drop-in)

```
---
name: entity-and-citation-rendering
description: Mark named entities and source citations distinctly in responses that use web search,
  so the frontend can render them as recognizable, previewable elements rather than plain text or b
  are links. Use only in responses that reference external search results.
---

## Entities
Mark a clear, well-known named entity (a specific person, company, place, product) the first time
it's mentioned in a response, so the interface can offer a preview. Don't mark generic nouns or ev
ery repeated mention of the same entity – once per entity per response is enough.

## Citations vs. links
Distinguish a citation (a source backing a specific factual claim) from an ordinary hyperlink (e.g
., "here's the docs page"). Attach a citation directly to the sentence or claim it supports, not a
s a generic list at the end, so the interface can place it inline.

## Rich media results
When search results are predominantly images or video, prefer surfacing them as a compact visual s
```

et rather than describing them purely in prose.

Fallback behavior

If entity or citation marking isn't confidently applicable, omit it rather than guessing – plain, correctly-formatted text is always safer than a wrong or misleading marker.

Workflow Router for Complex Requests

Why it matters

As Manthan Agent's tool surface grows (MCP tools, file generation, Python execution, web search, live previews), a single incoming message increasingly needs more than one capability chained together correctly. An explicit routing/decomposition step reduces wrong-tool selection and makes multi-step requests more reliable.

What “done” looks like

- A complex request is broken into sub-tasks before any tool is called.
- The right combination of tools/skills is selected deliberately, not by trial and error mid-reply.
- A brief self-check runs before the final answer is sent, catching obviously incomplete output.
- Which tools were used and why is loggable for debugging, without being shown as noise to the user.

How it fits Manthan Agent's architecture

This is a prompt-engineering and light-orchestration change more than a UI change. Since Manthan Agent already supports an "agent-swarm" style skill system, this slots in as a pre-step in that pipeline: (1) on receiving a message, run a short, cheap classification pass (can reuse a smaller/faster model) that outputs a structured plan — which skills/tools apply, in what order; (2) execute the plan, allowing normal tool-calling to still adapt mid-stream if the plan turns out wrong; (3) before sending the final reply, run a short self-check pass confirming the response actually addresses every part of the original request; (4) log the plan and tools used to your existing debug/telemetry surface, hidden from the end user.

Skill definition (drop-in)

```
---
name: request-router
description: Decompose an incoming request into sub-tasks and select which skills/tools apply before executing, then run a brief completeness check before the final reply. Use for requests that plausibly need more than one capability (e.g., research + document generation, or data analysis + chart creation).
---

## Step 1 – decompose
Identify the distinct parts of the request. A request to "research X and build me a slide deck" has two parts: research, then deck-building. Don't over-decompose simple, single-purpose requests – this step should be near-instant for anything straightforward.

## Step 2 – select
For each sub-task, pick the specific tool/skill that fits, rather than reaching for the first plausible one. Prefer tools that already have the needed data over ones that would require asking the user for information you can otherwise retrieve.

## Step 3 – execute
```

Run the plan, but don't treat it as rigid – if a tool call reveals the plan was wrong (e.g., a connected app isn't actually connected), adapt and continue rather than failing the whole request.

Step 4 – self-check

Before sending the final reply, verify every distinct part identified in Step 1 was actually addressed. If something was dropped, either complete it or explicitly tell the user what's missing and why – never silently omit part of a multi-part request.

Logging

Record which skills/tools were selected and used for each request in developer-facing logs only. This should never be surfaced to the end user as visible reasoning or tool-call noise.